

# 突破语义鸿沟：基于 LLM 技术的软件漏洞分析与挖掘技术发展现状调研报告

## 摘要

随着软件供应链复杂度的激增，传统基于规则的静态分析（SAST）与基于覆盖率的模糊测试（Fuzzing）技术在面对深层逻辑漏洞时逐渐触及瓶颈。近年来，大语言模型（LLM）的崛起为软件安全领域带来了范式转移。本文深入调研了 2023 至 2025 年间，基于 LLM 的漏洞挖掘技术从“静态代码分类”向“生成式模糊测试”与“自主智能体（Agent）”协同演进的历程。报告重点分析了 LLM 在通用模糊测试种子生成（如 Fuzz4All）、自动化渗透测试（如 PentestGPT）以及多语言漏洞基准测试中的实际表现。研究表明，虽然 LLM 在理解代码语义上超越了传统深度学习模型，但在处理跨文件上下文、降低误报率以及应对“幻觉”方面仍面临挑战。未来趋势将指向神经符号 AI 与多智能体协同攻防的深度融合。[\[1\]](#)

## 1. 引言

软件漏洞挖掘一直是网络安全攻防的核心命题。在过去的二十年里，该领域经历了从手工审计到自动化工具的演变。然而，以 Coverity 为代表的静态分析工具依赖预定义的漏洞模式（Pattern Matching），难以理解复杂的业务逻辑；以 AFL 为代表的模糊测试工具虽然能发现内存破坏类漏洞，但在面对高度结构化的输入（如编译器、解释器输入）时，往往因无法通过语法校验而效率低下。

2023 年以来，随着 GPT-4、Claude 3.5 Sonnet、CodeLlama 等大语言模型的爆发，安全研究界开始探索利用 LLM 的通用推理能力来解决上述难题。与早期的基于 BERT 的代码模型（如 CodeBERT）不同，生成式 LLM 不仅能识别漏洞，还能生成攻击载荷（Payload）、修复代码甚至编写模糊测试驱动。进入 2025 年，研究重心已从单纯的“零样本检测”转向“Agent 驱动的自动化攻防”，旨在构建能够像人类专家一样进行多步推理、工具调用和环境交互的智能系统。本文旨在系统梳理这一技术变革的现状、关键技术路径及面临的深层挑战。

## 2. 核心技术范式演进[2][3]

基于 LLM 的漏洞挖掘技术并非一蹴而就，其技术路线经历了三个显著的阶段：

### 2.1 从分类预测到上下文学习

早期的尝试（约 2022-2023 年）主要集中在微调（Fine-tuning）小参数量的 LLM，将其视为一个二分类器（Vulnerable/Safe）。然而，这种方法高度依赖标注数据的质量，且容易过拟合。随着模型能力的提升，主流技术转向了上下文学习（In-Context Learning）。通过在 Prompt 中嵌入少样本（Few-shot）漏洞示例，或者利用思维链（Chain-of-Thought, CoT）提示工程，模型能够在不更新权重的情况下，通过逻辑推理识别出未见过的漏洞模式。

### 2.2 生成式模糊测试（Generative Fuzzing）的崛起

这是目前最成功的应用方向之一。传统 Fuzzer 在生成符合特定语法（如 SQL、C++ 源码、XML）的测试用例时往往捉襟见肘。LLM 天生具备极其强大的语法生成能力。Xia 等人（ICSE 2024）提出的 Fuzz4All 框架标志着这一领域的成熟。该工具不再依赖特定语言的语法模板，而是利用 LLM 根据简短的输入描述自动生成多样的、覆盖率极高的模糊测试输入。LLM 不仅充当了“种子生成器”，还充当了“智能变异器”，当传统变异策略陷入死胡同时，LLM 能够根据代码路径约束生成能够突破校验的新输入。

### 2.3 自主智能体（Autonomous Agents）

2024-2025 年的前沿趋势是“安全智能体”。与单次问答不同，Agent 具备记忆、规划和工具使用能力。Deng 等人（USENIX Security 2024）发布的 PentestGPT 是该领域的里程碑之作。它通过三个交互模块（推理、生成、解析）模拟了人类渗透测试专家的思维过程：先分析目标，制定攻击计划，生成具体的 Nmap 或 SQLMap 命令，分析工具回显，再根据结果修正攻击策略。这种“ReAct”（Reasoning and Acting）模式使得 LLM 能够处理复杂的、需要多步交互的 Web

逻辑漏洞。

### 3. 技术现状与基准评估

#### 3.1 静态检测能力的虚与实

在静态分析领域，LLM 展现出了发现逻辑漏洞（Logic Bugs）的巨大潜力，这是传统工具的盲区。例如，权限校验缺失或业务流程绕过等问题，往往需要理解变量的语义命名和函数间的上下文关系，这正是 LLM 的强项。

然而，Tamberg 和 Bahsi（IEEE Access 2024）的基准测试研究揭示了当前技术的真实水平。他们对多个 SOTA 模型进行了大规模评估，结果显示：虽然 LLM 在召回率（Recall）上表现优异，能够发现传统工具漏掉的问题，但其精确率（Precision）往往较低，伴随着大量的误报（False Positives）。此外，LLM 在“语句级定位”（指出具体哪一行代码有问题）上的表现远不如其在函数级分类上的表现。这表明，模型可能更多是基于代码的整体“气味”（Look and Feel）来判断风险，而非进行了精确的数据流分析。

#### 3.2 漏洞修复与解释

LLM 在漏洞修复（APR）方面的表现令人印象深刻。对于缓冲区溢出、空指针解引用等常见 CWE 类型，LLM 生成的补丁通过率往往高于传统的基于搜索或约束求解的修复工具。更重要的是，LLM 能够用自然语言解释漏洞成因，这对于安全运营中心（SOC）的分析师来说极具价值，能够大幅降低人工复核的成本。

### 4. 当前面临的挑战与技术瓶颈

尽管基于 LLM 的漏洞挖掘技术发展迅猛，但要真正落地到工业级生产环境，仍面临数个“阿喀琉斯之踵”。

#### 4.1 数据集污染与记忆效应

这是一个在学术界被广泛讨论的危机。许多 LLM 在预训练阶段可能已经“看

过”了 GitHub 上的历史漏洞修复 Commit。因此，当研究者使用 BigVul、Devign 等经典公开数据集进行测试时，模型的高准确率可能源于“记忆”而非真正的“推理”。2025 年的新研究开始致力于构建去重、去时间重叠的“清洁”数据集（如基于 2024 年后的新 CVE），在这些新数据集上，LLM 的性能通常会有显著下降。

## 4.2 幻觉与误报

对于安全团队而言，高误报率是不可接受的。LLM 倾向于过度自信地将复杂的、非标准的代码结构标记为漏洞。例如，它可能会认为一个自定义的加密函数是不安全的，仅仅因为它不符合标准库的命名规范。目前的 Prompt 工程虽然引入了“请引用具体代码行”、“请给出置信度”等约束，但并未根除模型一本正经胡说八道的问题。

## 4.3 上下文窗口与跨文件分析

真实世界的漏洞往往是“幽灵”般的——定义在文件 A 中的变量，经过文件 B 的处理，最终在文件 C 中触发漏洞。尽管目前的模型支持 128k 甚至更长的 Context Window，但直接将整个代码仓（Repository）塞入 Context 中不仅成本高昂，而且会引入大量的干扰信息（Noise），导致模型的注意力分散。目前的解决方案主要依赖 RAG（检索增强生成）技术，即先通过关键词或向量检索找到相关代码片段再喂给 LLM，但这依赖于检索的准确性。

## 5. 结论与展望

综上所述，2025 年基于 LLM 的软件漏洞分析技术正处于从“验证可行性”向“解决工程化难题”跨越的关键时期。LLM 不再仅仅是一个静态的分类器，而是正在演变为能够主动探索、交互和利用的智能实体。

**未来展望：**

1. **神经符号 AI (Neuro-symbolic AI):** 为了解决幻觉问题，未来的系统将倾向于融合架构，即利用 LLM 进行语义理解和假设生成，然后生成查询语句驱动 CodeQL 等传统静态分析引擎进行精确的路径验证。
2. **多模态分析:** 结合代码文本与程序执行时的内存状态截图、控制流图 (CFG) 图像，让模型“看”到程序的运行时行为。
3. **本地化与隐私:** 随着端侧模型能力的提升，基于本地部署的专用安全 LLM 将成为企业 DevSecOps 流程中的标配，以解决代码隐私泄露的顾虑。

LLM 并未宣告传统安全技术的终结，而是为其插上了理解语义的翅膀。在未来的人机协同安全运营中，LLM 将扮演核心的“副驾驶”角色。

## 参考文献

[1] Xia, C. S., Paltenghi, M., Tian, J. L., Pradel, M., & Zhang, L. (2024). [\[2\]](#) Fuzz4All: Universal Fuzzing with Large Language Models. [\[2\]](#) [\[4\]](#)

[\[2\]](#) Deng, G., Liu, Y., Mayoral-Vilches, V., et al. (2024). [\[5\]](#) PentestGPT: Evaluating and Harnessing Large Language Models for Automated Penetration Testing.

[\[3\]](#) Tamberg, K., & Bahsi, H. (2024). Harnessing Large Language Models for Software Vulnerability Detection: A Comprehensive Benchmarking Study.